Disaster Recovery & Business Continuity

KVM High Availability, Backup Strategies & Automated Failover

Moving from VMware SRM to open-source DR doesn't mean losing protection. KVM provides multiple HA and DR options — from QCOW2 snapshots to Ceph cross-site mirroring to Pacemaker automated failover. This guide covers backup strategies, RPO/RTO planning, and ready-to-use runbook templates.

RPO/RTO | QCOW2 Snapshots | Ceph Mirroring | Pacemaker HA | Velero | Live Migration | Runbooks

DR Architecture Overview

RPO/RTO tiers, VMware DR costs vs KVM, and hyper2kvm's role in DR strategy.

DR Tier Definitions

Tier	Strategy	RPO	RTO	VMware Solution	KVM Solution	VMware Cost	KVM Cost
Tier 1	Cold Standby	24 hrs	24 hrs	Manual restore from backup	Restore QCOW2 from backup	$5K/yr	$0
Tier 2	Warm Standby	4 hrs	4 hrs	vSphere Replication	rsync + LVM snapshots	$25K/yr	$0
Tier 3	Hot Standby	15 min	15 min	SRM + vSphere Rep	Ceph RBD mirroring	$85K/yr	$0
Tier 4	Active-Active	~0	~0	SRM + Metro vSAN	Ceph stretch cluster	$200K+/yr	$0

VMware SRM vs KVM DR Cost Comparison (100 VMs)

VMware SRM Stack (Annual)

- vSphere Enterprise Plus: $49,400 (8 CPUs)
- SRM Standard: $5,975/CPU x 8 = $47,800
- vSphere Replication: $2,495/CPU x 8 = $19,960
- vSAN (DR site): $44,600
- DR site vCenter: $8,975 + $2,250 SnS
- Support & Maintenance: $32,000
- **Annual Total: $205,985**
- **3-Year Total: $617,955**

KVM DR Stack (Annual)

- RHEL subscriptions (optional): $13,000
- Ceph (included in RHEL or free): $0
- Pacemaker (included in RHEL): $0
- Velero (open source): $0
- BorgBackup / Restic (open source): $0
- hyper2kvm for initial migration: included*
- **Annual Total: $0-$13,000**
- **3-Year Total: $0-$39,000**

hyper2kvm's Role in DR

Initial migration: Convert VMware VMs to KVM for the DR site. **Ongoing:** Nightly automated pipeline exports changed VMDKs and converts to QCOW2 on KVM DR hosts. **Failover:** KVM DR VMs are pre-converted and ready to boot — RTO drops from hours (re-converting) to minutes (just boot).

Backup Strategies for KVM

QCOW2 snapshots, LVM snapshots, and dedicated backup tools — the 3-2-1 rule.

Snapshot Comparison

Method	Speed	Space	Live VM	Consistent	Best For
QCOW2 Internal	Instant	CoW delta	Yes	Crash-consistent	Quick rollback, testing, dev environments
QCOW2 External	Instant	New overlay	Yes	Crash-consistent	Production snapshots, backup integration
LVM Snapshot	Instant	CoW delta	Yes	Crash-consistent	Block-level backup, LVM thin pools
Ceph RBD Snapshot	Instant	CoW delta	Yes	Crash-consistent	Distributed storage, cross-site replication
QEMU Guest Agent	Seconds	N/A (freeze)	Yes	App-consistent	Combined with any snapshot for consistency
Full VM Export	Minutes	Full copy	Offline	Full	Archival, compliance, disaster recovery

Backup Commands

QCOW2 Snapshots with Guest Agent

```
# Freeze guest filesystem (app-consistent)
virsh domfsfreeze db-server

# Create external snapshot
virsh snapshot-create-as db-server \
  snap-$(date +%Y%m%d) \
  --disk-only --atomic \
  --quiesce

# Backup the base image (snapshot active)
cp /var/lib/libvirt/images/db-server.qcow2 \
   /backup/db-server-$(date +%Y%m%d).qcow2

# Commit snapshot back (merge overlay)
virsh blockcommit db-server vda \
  --active --pivot
```

BorgBackup (Deduplicated)

```
# Initialize backup repository
borg init --encryption=repokey \
  /backup/borg-repo

# Backup VM disk with deduplication
virsh domfsfreeze db-server
borg create --progress --stats \
  /backup/borg-repo::db-server-{now} \
  /var/lib/libvirt/images/db-server.qcow2
virsh domfsthaw db-server

# Prune old backups (keep 7 daily, 4 weekly)
borg prune /backup/borg-repo \
  --keep-daily=7 --keep-weekly=4

# Restore
borg extract /backup/borg-repo::db-server-2026-03-23
cp db-server.qcow2 /var/lib/libvirt/images/
```

```
# Thaw guest filesystem
virsh domfsthaw db-server
```

Restic (Cloud-Compatible)

```
# Initialize (local, S3, SFTP, etc.)
restic init --repo /backup/restic-repo

# Backup with progress
restic backup \
  /var/lib/libvirt/images/db-server.qcow2 \
  --repo /backup/restic-repo

# To S3 (air-gapped: use MinIO)
export RESTIC_REPOSITORY=s3:minio:9000/backups
restic backup /var/lib/libvirt/images/

# Restore specific snapshot
restic restore latest --target /restore/
# Or specific: restic restore abc123
```

3-2-1 Rule Implementation

- **3 copies:** Live disk + local backup + remote
- **2 media types:** NVMe (live) + HDD (backup) + tape/S3
- **1 offsite:** Restic to remote S3 or SFTP server
- **Schedule:** Hourly snapshots, nightly full backup
- **Retention:** 24 hourly, 7 daily, 4 weekly, 12 monthly
- **Testing:** Monthly restore drill (random VM)
- **Monitoring:** Alert if backup age > 25 hours
- **Encryption:** All backups encrypted at rest

Ceph RBD Mirroring for DR

Cross-site Ceph replication for near-zero RPO disaster recovery.

Mirroring Modes

Mode	RPO	Bandwidth	How It Works	Pros	Cons
Journal-based	Seconds	High (continuous)	Every write journaled and replayed at DR site	Near-zero RPO, point-in-time recovery	2x write latency, requires stable WAN link
Snapshot-based	5-15 min	Periodic bursts	Periodic snapshots synced as deltas	Lower bandwidth, tolerates WAN instability	RPO = snapshot interval

Setup Cross-Site Ceph Mirroring

```
# On PRIMARY site
# Enable mirroring on the VM pool
rbd mirror pool enable vm-pool image

# Enable mirroring on specific image
rbd mirror image enable vm-pool/db-server snapshot
# Or journal mode: rbd mirror image enable vm-pool/db-server journal

# Create snapshot schedule (every 15 minutes)
rbd mirror snapshot schedule add --pool vm-pool --image db-server 15m

# On SECONDARY (DR) site
# Bootstrap peer connection
rbd mirror pool peer bootstrap create --site-name primary vm-pool > token.txt
# Copy token.txt to DR site
rbd mirror pool peer bootstrap import --site-name secondary vm-pool token.txt

# Start rbd-mirror daemon on DR site
systemctl enable --now ceph-rbd-mirror@admin

# Verify mirroring status
rbd mirror pool status vm-pool
# health: OK
# images: 50 total
#     50 replaying
```

Failover & Failback

DR Failover Procedure

- **Step 1:** Verify primary is down (avoid split-brain)
- **Step 2:** Promote DR images: `rbd mirror image promote --force vm-pool/db-server`
- **Step 3:** Map RBD to KVM: `rbd map vm-pool/db-server`
- **Step 4:** Start VM: `virsh start db-server`
- **Step 5:** Update DNS records to DR site IPs
- Total failover time: 5-15 minutes (automated)
- Script: `ceph-failover.sh --pool vm-pool --promote-all`

Failback to Primary

- **Step 1:** Repair primary site Ceph cluster
- **Step 2:** Demote DR images: `rbd mirror image demote vm-pool/db-server`
- **Step 3:** Re-sync to primary (delta only)
- **Step 4:** Promote primary images
- **Step 5:** Shutdown DR VMs, start primary VMs
- **Step 6:** Update DNS back to primary IPs
- Resync time: depends on delta size (typically 1-4 hours)

KVM Live Migration for HA

Move running VMs between hosts with sub-100ms downtime.

Live Migration Prerequisites

Requirement	Details	Check Command	Common Issue
Shared Storage	NFS, Ceph RBD, GlusterFS, or iSCSI	`df -h /var/lib/libvirt/images`	Local disk = no live migration
Same CPU family	Source and dest must have compatible CPU	`virsh capabilities | grep model`	AMD→Intel fails; use host-model
Network connectivity	Migration port (TCP 49152+) open between hosts	`nc -zv dest-host 49152`	Firewall blocking migration port
libvirt version match	Same major version on both hosts	`virsh version`	Version mismatch = XML incompatibility
SELinux context	VM disk labels must match on both hosts	`ls -Z /var/lib/libvirt/images/`	svirt_image_t mismatch

Live Migration Commands

```
# Basic live migration
virsh migrate --live --verbose db-server qemu+ssh://kvm-host-02/system

# With bandwidth limit (avoid saturating network)
virsh migrate --live --bandwidth 500 db-server qemu+ssh://kvm-host-02/system
# 500 MiB/s = leaves room for production traffic on 10GbE

# Tunneled migration (encrypted, no extra ports)
virsh migrate --live --tunnelled db-server qemu+ssh://kvm-host-02/system

# With auto-convergence (for busy VMs that won't converge)
virsh migrate --live --auto-converge db-server qemu+ssh://kvm-host-02/system
# Throttles vCPU to reduce dirty page rate

# Post-copy (for very large memory VMs)
virsh migrate --live --postcopy db-server qemu+ssh://kvm-host-02/system
# Moves VM first, fetches memory pages on demand

# Monitor migration progress
virsh domjobinfo db-server
# Time elapsed:    45s
# Data remaining:  128 MiB
# Memory remaining: 64 MiB
# Expected downtime: 50ms
```

Performance: Live Migration Downtime

Typical: 50-200ms downtime (TCP connections survive). **Large memory VMs (256GB+):** Use auto-converge or post-copy to guarantee convergence. **Best practice:** Dedicated migration network (separate VLAN/NIC) with 10GbE minimum. Migration of a 64GB RAM VM over 10GbE takes ~30 seconds total with <100ms downtime.

Automated Failover with Pacemaker

Pacemaker/Corosync cluster provides automatic VM failover on host failure.

Pacemaker HA Cluster Setup

```
# Install on all cluster nodes
dnf install pacemaker corosync pcs fence-agents-all

# Initialize cluster (2 or 3 nodes)
pcs host auth kvm-host-01 kvm-host-02 kvm-host-03
pcs cluster setup ha-cluster kvm-host-01 kvm-host-02 kvm-host-03
pcs cluster start --all && pcs cluster enable --all

# Configure fencing (STONITH) — REQUIRED for production
# IPMI fencing
pcs stonith create fence-host01 fence_ipmilan \
  ipaddr=10.0.0.101 login=admin passwd=secret lanplus=1 \
  pcmk_host_list=kvm-host-01

# Add VM as cluster resource
pcs resource create vm-db-server VirtualDomain \
  hypervisor="qemu:///system" \
  config="/etc/libvirt/qemu/db-server.xml" \
  migration_transport="ssh" \
  meta allow-migrate=true \
  op start timeout=120 \
  op stop timeout=120 \
  op monitor timeout=30 interval=10

# Prefer primary host, allow failover to others
pcs constraint location vm-db-server prefers kvm-host-01=100
pcs constraint location vm-db-server prefers kvm-host-02=50
```

Cluster Topology Options

2-Node Cluster

- Simplest HA configuration
- Requires quorum device (qdevice) to avoid split-brain
- `pcs quorum device add model net host=qdevice-host`
- Active-passive: VMs run on node 1, failover to node 2
- Capacity: N+1 (node 2 must hold all VMs from node 1)

3-Node Cluster

- Native quorum (no external qdevice needed)
- Survives 1 node failure with VMs redistributed
- Active-active: VMs spread across all 3 nodes
- Capacity: N+1 (each node holds ~33% of VMs normally)
- On failure: 2 remaining nodes each take 50%

- Cost: 2 servers + 1 lightweight qdevice (can be a VM)
- Best for: small deployments, 10-30 VMs

- Aligns with Ceph minimum (3 nodes, replication factor 3)
- Best for: production, 30-100 VMs

Critical: Always Configure Fencing (STONITH)

Without STONITH, a node that appears down but is actually running causes split-brain — two nodes running the same VM simultaneously, leading to data corruption. Use IPMI, iLO, DRAC, or cloud fencing agents. **Never run `pcs property set stonith-enabled=false` in production.**

KubeVirt DR with Velero

Backup and restore KubeVirt VMs across Kubernetes clusters.

Velero Setup for KubeVirt

```
# Install Velero with CSI snapshot support
velero install \
  --provider aws \
  --bucket kubevirt-backups \
  --secret-file ./credentials-velero \
  --use-volume-snapshots=true \
  --features=EnableCSI \
  --plugins velero/velero-plugin-for-aws:v1.8.0,velero/velero-plugin-for-csi:v0.6.0

# Backup a KubeVirt VM (includes PVCs)
velero backup create db-server-backup \
  --include-namespaces production \
  --selector kubevirt.io/vm=db-server \
  --snapshot-volumes=true

# Schedule periodic backups (every 6 hours)
velero schedule create kubevirt-backup \
  --schedule="0 */6 * * *" \
  --include-namespaces production \
  --selector app.kubernetes.io/part-of=kubevirt \
  --snapshot-volumes=true \
  --ttl 168h  # 7-day retention
```

DR Workflows

Cross-Cluster Restore

- **Step 1:** Install Velero on DR cluster (same bucket)
- **Step 2:** List available backups: `velero backup get`
- **Step 3:** Restore: `velero restore create --from-backup db-server-backup`
- **Step 4:** VM + PVCs + ConfigMaps restored automatically
- **Step 5:** Start VM: `virtctl start db-server`
- RTO: 10-30 minutes (depends on PVC size)
- Works across cloud providers (AWS→on-prem, etc.)

DR Testing with Staging

- Restore to staging namespace (non-destructive test)
- `velero restore create --from-backup prod-backup --namespace-mappings production:staging`
- Verify VM boots and applications respond
- Run smoke tests against staging VMs
- Delete staging restore when done
- Schedule monthly: automated DR drill

- Report: pass/fail, RTO measured, issues found

CSI Volume Snapshots

- Ceph RBD: `VolumeSnapshotClass: rook-ceph-rbd`
- Longhorn: `VolumeSnapshotClass: longhorn`
- Snapshots are crash-consistent by default
- For app-consistent: use pre-backup hooks
- `velero backup create --pre-hook="virtctl fsfreeze db-server"`
- Incremental: only changed blocks transferred

Velero + KubeVirt RPO/RTO

- **RPO:** = backup schedule interval (6h default)
- **RTO:** = restore time (10-30 min for typical VM)
- Smaller RPO: increase schedule frequency
- Smaller RTO: pre-provision PVCs at DR site
- Combine with Ceph mirroring for RPO < 1 minute
- Cost: S3 storage only ($0.023/GB/month)

DR Runbook Templates

Step-by-step procedures for four DR scenarios.

Runbook 1: Single VM Recovery

- **Trigger:** VM disk corruption, accidental deletion
- **RTO Target:** 30 minutes
- 1. Identify affected VM and failure type
- 2. Stop corrupted VM: `virsh destroy <vm>`
- 3. Restore from latest backup:
  `borg extract /backup/borg-repo::<vm>-latest`
- 4. Copy restored disk to libvirt images directory
- 5. Start VM: `virsh start <vm>`
- 6. Verify application health and connectivity
- 7. Document: timestamp, root cause, recovery time
- **Escalation:** If backup corrupt → restore from secondary

Runbook 2: Full Site Failover

- **Trigger:** Primary site unreachable > 15 minutes
- **RTO Target:** 30 minutes
- 1. Confirm primary is down (avoid false positive)
- 2. Notify stakeholders: "DR failover initiated"
- 3. Promote Ceph DR images:
  `for img in $(rbd ls vm-pool); do rbd mirror image promote --force vm-pool/$img; done`
- 4. Start VMs in dependency order (DB → App → Web)
- 5. Update DNS to DR site IPs (TTL was pre-lowered)
- 6. Verify all services responding
- 7. Notify stakeholders: "Failover complete"
- **Escalation:** If Ceph images stale → restore from Velero

Runbook 3: Ransomware Recovery

- **Trigger:** Ransomware detected on VMs
- **RTO Target:** 4 hours
- 1. Isolate affected VMs: disconnect network immediately
- 2. Snapshot affected VMs (evidence preservation)
- 3. Identify infection timestamp from logs
- 4. Restore from backup BEFORE infection:
  `borg extract /backup/borg-repo::<vm>-2026-03-20`
- 5. Rebuild on clean host (re-image host OS if needed)
- 6. Restore VMs on clean host, isolated network
- 7. Scan restored VMs before reconnecting to network
- **Critical:** Do NOT boot infected snapshots on production

Runbook 4: DC Evacuation

- **Trigger:** Physical threat (fire, flood, power failure)
- **RTO Target:** 1 hour
- 1. If time permits: graceful VM shutdown + final Ceph sync
- 2. If no time: failover immediately to DR site
- 3. Promote all Ceph DR images (automated script)
- 4. Start critical VMs first (Tier 1: DB, AD, DNS)
- 5. Start remaining VMs in priority order
- 6. Update external DNS to DR site
- 7. Notify customers of failover and any data loss
- **Pre-staged:** DR VMs pre-defined in Pacemaker cluster

Cost Analysis & Implementation

VMware SRM vs open-source KVM DR stack — 3-year TCO for 100 VMs.

3-Year TCO Comparison

Cost Category	VMware SRM	KVM (Ceph + Pacemaker)	Notes
Software licenses (3 yr)	$617,955	$0-$39,000	KVM: optional RHEL subs
DR site hardware	$120,000	$120,000	Same hardware either way
Implementation labor	$80,000	$60,000	KVM: simpler architecture
Training	$15,000	$20,000	KVM: new skills for VMware team
Annual operations (3 yr)	$180,000	$150,000	KVM: fewer moving parts
Annual testing/drills (3 yr)	$30,000	$30,000	Same effort required
3-Year Total	**$1,042,955**	**$380,000-$419,000**	**60% savings with KVM**

Implementation Timeline

Week	Activity	Deliverable
1-2	Assessment: inventory VMs, classify RPO/RTO tiers, select DR strategy	DR plan document, VM classification matrix
3-4	Infrastructure: deploy DR site servers, install OS, configure Ceph cluster	3-node Ceph cluster operational, storage pools created
5-6	Migration: use hyper2kvm to convert VMware VMs to KVM on DR site	All VMs converted, boot-tested on DR hosts
7	HA setup: configure Pacemaker cluster, fencing, VM resources	Pacemaker cluster with automatic failover tested
8	Replication: enable Ceph mirroring, configure backup schedules	Ceph mirroring active, Borg/Restic backups running
9	Testing: full DR drill — simulate primary failure, measure RTO	DR drill report, measured RTO vs target
10	Documentation: finalize runbooks, train operations team	4 runbooks, trained team, monitoring dashboards

Open-Source DR Saves $623K Over 3 Years While Matching VMware SRM Capabilities

Ceph RBD mirroring achieves the same RPO as vSphere Replication (seconds with journal mode). Pacemaker provides equivalent automatic failover to VMware HA. Live migration matches vMotion. The only difference: $0 in software licensing vs $618K for VMware SRM. hyper2kvm bridges the gap — convert once, protect forever.